Algorithm 1: Overall Agentic Document Extraction Framework

Input: D: Input document (PDF, DOCX, Image)
Output: M: Metadata file, J: Structured JSON document

Receive input document D;
Invoke **Router Agent**[2];
Invoke **Document Classification Agent**[3];
if *Document requires OCR* **then**
    Invoke **OCR Quality Assessment Agent**[4];
    if *OCR quality* $< \tau$ **then**
        Invoke **Image Enhancement Agent**[5];
else
    Continue document processing;
Invoke **Layout Detection Agent**[6];
Invoke **Reading Order Agent**[7];
Invoke **Section Identification Agent**[8];
foreach *identified section* S_i **do**
    Invoke **Context Memory Agent**[9];
    if *Table exists* **then**
        Invoke **Table Analysis Agent**[10];
    if *Figure exists* **then**
        Invoke **Figure Understanding Agent**[11];
    if *Equation exists* **then**
        Invoke **Equation Analysis Agent**[12];
    Invoke **Internal Citation Extraction Agent**[13];
    Invoke **Cross-Reference Validation Agent**[14];
    Invoke **Entity Extraction Agent**[15];
    Invoke **Abbreviation Resolution Agent**[16];
    Invoke **Text Summarization Agent**[17];
    Invoke **Hallucination Detection Agent**[18];
    Invoke **Confidence Scoring Agent**[19];
Invoke **Metadata Fusion & JSON Appending Agent**[20];
Invoke **Global Validation Agent**[21];
Export
    Metadata file (M);
    Structured JSON file (J);
    Validation Report;
Return (M, J);

Algorithm 2: Router Agent

Input: D: Input document
Output: P: Parsed document stream, T: Document type

```
Receive input document D;
Validate document accessibility;
if Document is inaccessible then
    Return error message;
else
    Continue processing;
Determine file extension and MIME type;
if Extension = PDF then
    T ← PDF;
    Invoke PDF Parser;
else
    if Extension = DOCX then
        T ← DOCX;
        Invoke DOCX Parser;
    else
        if Extension ∈ {PNG, JPG, JPEG, TIFF, BMP} then
            T ← IMAGE;
            Invoke OCR Pipeline;
        else
            Reject unsupported document;
            Exit;
Extract raw document content;
Normalize character encoding to UTF-8;
Remove unsupported embedded objects;
Extract document properties
    Title;
    Author;
    Creation Date;
    Number of Pages;
    Language;
    File Size;
Generate parsed document stream P;
Forward (P, T) to Document Classification Agent;
Return (P, T);
```

Algorithm 3: Document Classification Agent

Input: P: Parsed document stream
Output: C: Document category

Receive parsed document stream P;
Extract document features
    Document title;
    Table of contents (if available);
    List of sections and subsections;
    Headings and subheadings;
    Keywords and domain-specific terms;
    Document metadata;
    Tables and figures count;
    Page layout statistics;
Extract semantic embeddings using a language model;
Construct feature vector $F \leftarrow \{f_1, f_2, \ldots, f_n\}$;
Compare feature vector with predefined document classes
    Standard Operating Procedure (SOP);
    User Manual;
    Technical Report;
    Installation Guide;
    Maintenance Manual;
    Machine Log / System Log;
    Incident Report;
    Test Report;
    Research Document;
    Policy or Compliance Document;
    Invoice / Form;
    Miscellaneous;
Compute confidence score for each document class

$$Conf(C_i) = Similarity(F, C_i)$$

Determine predicted category

$$C = \arg\max_i Conf(C_i)$$

if $Conf(C) < \tau$ **then**
    Assign category as Unknown;
    Flag document for manual verification;
else
    Accept predicted document category;
Store document category and confidence score in metadata;
Forward document to the corresponding processing workflow;
Return document category C;

Algorithm 4: OCR Quality Assessment Agent

Input: I: Input document image(s)
Output: Q: OCR Quality Report, I': Validated image(s)

Receive input image(s) I;
Initialize quality report $Q \leftarrow \{\}$;
foreach *page image* I_i **do**
    Compute image quality metrics
        Image resolution;
        Brightness;
        Contrast;
        Sharpness;
        Noise level;
        Blur score;
        Skew angle;
        Rotation angle;
        Compression artifacts;
        OCR confidence score;
    Detect image defects
        Missing regions;
        Broken characters;
        Shadow regions;
        Watermarks;
        Low illumination;
        Background noise;
    Compute overall quality score

$$Q_i = \sum_{k=1}^{n} w_k m_k$$

    where m_k denotes the normalized quality metric and w_k represents its corresponding weight.
    if $Q_i \geq \tau_{quality}$ **then**
        Mark page as VALID;
        Store quality statistics;
    else
        Mark page as LOW QUALITY;
        Forward page to Image Enhancement Agent;
    Update quality report Q;
Compute overall document quality score;
Store page-wise confidence values;
Return (Q, I');

Algorithm 5: Image Enhancement Agent

Input: I: Low-quality document image(s)
Output: E: Enhanced image(s)

Receive image(s) requiring enhancement;
foreach *image* I_i **do**
    Estimate degradation type
        Blur;
        Noise;
        Low contrast;
        Skew;
        Rotation;
        Shadow;
        Uneven illumination;
        Low resolution;
    Apply preprocessing
    if *Noise detected* **then**
        Perform image denoising;
    if *Blur detected* **then**
        Apply image sharpening or deblurring;
    if *Low contrast detected* **then**
        Perform contrast enhancement;
    if *Low illumination detected* **then**
        Apply adaptive brightness correction;
    if *Image is skewed* **then**
        Estimate skew angle;
        Deskew image;
    if *Image is rotated* **then**
        Correct orientation;
    if *Resolution is below threshold* **then**
        Apply super-resolution enhancement;
    if *Background artifacts detected* **then**
        Remove background noise;
        Suppress shadows;
    Normalize image
        Intensity normalization;
        Color normalization;
        Resolution normalization;
    Generate enhanced image E_i;
Invoke OCR engine on enhanced images;
Measure OCR confidence improvement

$$\Delta C = C_{after} - C_{before}$$

if $\Delta C > 0$ **then**
    Accept enhanced image;
else
    Restore original image;
    Flag for manual inspection;
Return enhanced image set E;

Algorithm 6: Layout Detection Agent

Input: P: Parsed document stream
Output: L: Document layout representation

Receive parsed document stream P;
Initialize layout map $L \leftarrow \{\}$;
foreach *page* p_i *in* P **do**
    Detect page dimensions and orientation;
    Identify structural regions
        Header;
        Footer;
        Body text;
        Margins;
        Page number;
        Columns;
        Text blocks;
        Whitespace regions;
    Detect document elements
        Titles;
        Headings;
        Subheadings;
        Paragraphs;
        Lists;
        Tables;
        Figures;
        Captions;
        Equations;
        Algorithms;
        References;
        Footnotes;
    Determine page layout
    if *Single-column layout* **then**
        Assign layout type = Single Column;
    else
        if *Double-column layout* **then**
            Assign layout type = Double Column;
        else
            Assign layout type = Multi-column;
    Compute bounding box for each detected element

$$B = (x, y, w, h)$$

    Store
        Element type;
        Bounding box;
        Page number;
        Reading priority;
    Update layout map L;
Validate layout consistency
    No overlapping regions;
    Valid reading regions;
    Caption alignment;
    Table alignment;
    Figure alignment;
if *Layout validation succeeds* **then**
    Approve layout map;
else
    Flag page for manual review or reprocessing;
Forward layout map to Reading Order Agent;
Return layout representation L;

Algorithm 7: Reading Order Agent

Input: L: Document layout representation
Output: R: Ordered document structure

Receive layout representation L;
Initialize reading sequence $R \leftarrow \{\}$;
foreach *page* p_i *in* L **do**
    Identify all layout elements
        Title;
        Header;
        Section headings;
        Subheadings;
        Paragraphs;
        Lists;
        Tables;
        Figures;
        Captions;
        Equations;
        Footnotes;
        References;
    Determine page layout type
    if *Single-column* **then**
        Sort elements from top-to-bottom;
    else
        if *Double-column* **then**
            Sort left column from top-to-bottom;
            Sort right column from top-to-bottom;
            Merge both columns into logical reading order;
        else
            Apply graph-based reading order estimation;
    Associate captions with their corresponding tables and figures;
    Associate equations with surrounding explanatory text;
    Associate footnotes with referenced content;
    Resolve cross-page reading continuity
        Split paragraphs;
        Multi-page tables;
        Multi-page figures;
    Assign reading priority

$$Priority(e_i) = f(y, x, \text{layout})$$

    Sort document elements according to priority;
    Append ordered elements to reading sequence R;
Validate reading sequence
    No duplicated elements;
    No missing elements;
    Correct column transitions;
    Valid caption associations;
    Preserve document hierarchy;
if *Validation succeeds* **then**
    Approve reading sequence;
else
    Flag page for manual inspection;
Forward ordered structure to Section Identification Agent;
Return ordered document structure R;

Algorithm 8: Section Identification Agent

Input: R: Ordered document structure
Output: S: Hierarchical section representation

```
Receive ordered document structure R;
Initialize section hierarchy S ← {};
Initialize current section and subsection as NULL;
foreach document element e_i ∈ R do
    Identify element type
        Document title;
        Section heading;
        Subsection heading;
        Paragraph;
        Table;
        Figure;
        Equation;
        Reference;
    if e_i is a Section heading then
        Create new section node;
        Set current section;
    else
        if e_i is a Subsection heading then
            Create subsection under current section;
            Set current subsection;
        else
            Append element to active section/subsection;
    Associate extracted objects with the current section
        Tables and captions;
        Figures and captions;
        Equations;
        References;
    Update section metadata
        Section ID;
        Title;
        Page range;
        Content statistics;
Validate hierarchy
    Section continuity;
    Hierarchy consistency;
    Object association;
    No orphan content;
if Validation succeeds then
    Approve section hierarchy;
else
    Flag inconsistencies for correction;
Forward hierarchy to Context Memory Agent;
Return hierarchical section representation S;
```

Algorithm 9: Context Memory Agent

Input: S: Hierarchical document section
Output: M_s: Section memory representation

Receive section S;
Initialize memory repository $M_s \leftarrow \{\}$;
Compute token length

$$L = \text{TokenCount}(S)$$

if $L \leq ContextLimit$ **then**
    Store complete section in memory;
    Generate semantic embedding;
    Associate section metadata;
else
    Partition section using semantic chunking

$$S = \{C_1, C_2, \ldots, C_n\}$$

    foreach *chunk* C_i **do**
        Generate chunk identifier;
        Compute semantic embedding;
        Extract contextual metadata
            Chunk ID;
            Parent Section;
            Previous Chunk;
            Next Chunk;
            Page Range;
            Token Count;
        Generate chunk summary;
        Extract
            Key concepts;
            Entities;
            Tables;
            Figures;
            Equations;
            References;
        Store chunk representation in memory;
    Construct chunk relationship graph

$$G = (V, E)$$

    where
        V represents document chunks;
        E represents semantic relationships;
    Generate section memory summary;
    Store section embedding;
Create retrieval index for memory repository;
Validate memory consistency
    No missing chunks;
    Correct chunk sequence;
    Valid semantic links;
    Consistent metadata;
if *Validation succeeds* **then**
    Approve memory representation;
else
    Reconstruct invalid memory entries;
    Revalidate;
Forward memory representation to downstream agents
    Table Analysis Agent;
    Figure Analysis Agent;
    Equation Analysis Agent;
    Text Summarization Agent;
Return section memory representation M_s;

Algorithm 10: Table Analysis Agent

Input: S: Section content
Output: T: Structured table representation

Receive section content S;
Initialize table repository $T \leftarrow \{\}$;
Detect all tables within the section;
foreach *table* t_i **do**
    Extract table properties
        Table number;
        Caption;
        Page number;
        Bounding region;
    Extract table structure
        Headers;
        Rows;
        Columns;
        Merged cells;
        Units;
    Normalize table values;
    Identify key attributes and relationships;
    Generate semantic table description;
    Summarize the table content;
    Associate table with parent section;
    Update Metadata
        Table ID;
        Caption;
        Summary;
        Keywords;
    Store structured table in JSON
        Headers;
        Rows;
        Description;
        Source page;
Validate extracted tables
    Header consistency;
    Missing cells;
    Caption association;
    Structural integrity;
if *Validation succeeds* **then**
    Approve extracted tables;
else
    Flag table for reprocessing;
Forward table repository to Metadata Fusion Agent;
Return structured table representation T;

Algorithm 11: Figure Understanding Agent

Input: S: Section content
Output: F: Structured figure representation

Receive section content S;
Initialize figure repository $F \leftarrow \{\}$;
Detect all figures within the section;
foreach *figure* f_i **do**
    Extract figure properties
        Figure number;
        Caption;
        Page number;
        Bounding region;
    Identify figure type
        Flowchart;
        Architecture diagram;
        Block diagram;
        Chart/Graph;
        Image;
        Pipeline;
        Network topology;
        Miscellaneous;
    Extract figure components
        Objects;
        Labels;
        Legends;
        Text annotations;
        Connectors/Arrows;
    Analyze relationships among detected components;
    Generate detailed figure description;
    Infer workflow or step-by-step process;
    Associate figure with parent section;
    Update Metadata
        Figure ID;
        Caption;
        Summary;
        Keywords;
    Store structured figure in JSON
        Objects;
        Relationships;
        Workflow;
        Description;
        Source page;
Validate extracted figures
    Caption association;
    Legend consistency;
    Object completeness;
    Workflow integrity;
if *Validation succeeds* **then**
    Approve extracted figures;
else
    Flag figure for reprocessing;
Forward figure repository to Metadata Fusion Agent;
Return structured figure representation F;

Algorithm 12: Equation Analysis Agent

Input: S: Section content
Output: E: Structured equation representation

Receive section content S;
Initialize equation repository $E \leftarrow \{\}$;
Detect all equations within the section;
foreach *equation* e_i **do**
    Extract equation properties
        Equation number;
        LaTeX representation;
        Page number;
        Bounding region;
    Identify equation type
        Algebraic;
        Statistical;
        Optimization;
        Differential;
        Logical;
        Matrix;
        Miscellaneous;
    Extract equation components
        Variables;
        Constants;
        Operators;
        Functions;
        Constraints;
    Generate semantic description of the equation;
    Identify variable definitions from surrounding text;
    Associate equation with parent section;
    Update Metadata
        Equation ID;
        Equation type;
        Summary;
        Variables;
    Store structured equation in JSON
        LaTeX expression;
        Variables;
        Description;
        Source page;
Validate extracted equations
    Syntax correctness;
    Variable consistency;
    Equation-reference mapping;
    LaTeX integrity;
if *Validation succeeds* **then**
    Approve extracted equations;
else
    Flag equation for reprocessing;
Forward equation repository to Metadata Fusion Agent;
Return structured equation representation E;

Algorithm 13: Internal Citation Extraction Agent

Input: S: Hierarchical document section
Output: C: Internal citation repository

Receive section hierarchy S;
Initialize citation repository $C \leftarrow \{\}$;
foreach *section* S_i **do**
    Scan text for internal references;
    Identify citation types
        Section references;
        Subsection references;
        Figure references;
        Table references;
        Equation references;
        Algorithm references;
        Appendix references;
        Page references;
    Extract citation identifier and referenced object;
    Resolve citation target within the document;
    Verify referenced object exists;
    Associate citation with source section;
    Store citation information
        Citation ID;
        Citation type;
        Source section;
        Target object;
        Target page;
    Update Metadata
        Referenced figures;
        Referenced tables;
        Referenced equations;
        Referenced sections;
    Store citation mapping in JSON;
Validate extracted citations
    Broken references;
    Duplicate citations;
    Invalid numbering;
    Missing target objects;
if *Validation succeeds* **then**
    Approve citation repository;
else
    Flag unresolved citations for correction;
Forward citation repository to Cross-Reference Agent;
Return internal citation repository C;

Algorithm 14: Cross-Reference Validation Agent

Input: C: Internal citation repository, S: Document hierarchy
Output: R: Validated cross-reference repository

Receive citation repository C and document hierarchy S;
Initialize validated repository $R \leftarrow \{\}$;
foreach *citation* $c_i \in C$ **do**
    Identify referenced object
        Section;
        Subsection;
        Figure;
        Table;
        Equation;
        Algorithm;
        Appendix;
    Search corresponding object in document hierarchy;
    if *Referenced object exists* **then**
        Validate
            Reference number;
            Section hierarchy;
            Page location;
            Object uniqueness;
        Create bidirectional mapping
        Source $\leftrightarrow$ Target;
        Update Metadata and JSON;
        Store validated reference in R;
    else
        Flag unresolved reference;
        Log validation error;
        Suggest nearest matching object;
Validate document consistency
    Broken references;
    Duplicate references;
    Invalid numbering;
    Orphan objects;
    Circular references;
Generate cross-reference validation report;
if *Validation succeeds* **then**
    Approve document references;
else
    Flag document for manual review;
Forward validated repository to Metadata Fusion Agent;
Return validated cross-reference repository R;

Algorithm 15: Entity Extraction Agent

Input: S: Hierarchical document section
Output: E: Structured entity repository

Receive section hierarchy S;
Initialize entity repository $E \leftarrow \{\}$;
foreach *section* S_i **do**
    Extract textual content;
    Identify candidate entities using NER and LLM;
    Classify entities into
        Person;
        Organization;
        Location;
        Software / Framework;
        Hardware;
        Dataset;
        Model;
        API / Tool;
        Standard / Protocol;
        Domain-specific terms;
    Normalize entity names;
    Remove duplicate entities;
    Extract entity attributes and surrounding context;
    Associate entities with parent section;
    Generate semantic description for each entity;
    Store entity information
        Entity ID;
        Entity type;
        Entity name;
        Description;
        Section ID;
        Page number;
    Update Metadata and JSON;
Validate extracted entities
    Duplicate entities;
    Missing descriptions;
    Invalid classifications;
    Context consistency;
if *Validation succeeds* **then**
    Approve entity repository;
else
    Flag entities for re-extraction;
Forward entity repository to Relationship Extraction Agent;
Return structured entity repository E;

Algorithm 16: Abbreviation Resolution Agent

Input: S: Hierarchical document section
Output: A: Abbreviation repository

Receive section hierarchy S;
Initialize abbreviation repository $A \leftarrow \{\}$;
foreach *section* S_i **do**
    Scan textual content for abbreviations;
    Detect abbreviation-definition pairs
        Full Term (ABBR)
    or
        ABBR (Full Term)
    Extract
        Abbreviation;
        Expanded form;
        Section ID;
        Page number;
    Normalize abbreviation format;
    Resolve repeated abbreviations using existing repository;
    Verify abbreviation consistency across sections;
    Generate semantic description for the abbreviation;
    Update Metadata
        Abbreviation;
        Expanded form;
        Occurrences;
    Store abbreviation mapping in JSON;
Validate abbreviation repository
    Duplicate abbreviations;
    Conflicting definitions;
    Undefined abbreviations;
    Unused abbreviations;
if *Validation succeeds* **then**
    Approve abbreviation repository;
else
    Flag inconsistencies for manual review;
Forward abbreviation repository to Metadata Fusion Agent;
Return abbreviation repository A;

Algorithm 17: Text Summarization Agent

Input: S: Hierarchical document section
Output: T: Section summary repository

Receive section hierarchy S;
Initialize summary repository $T \leftarrow \{\}$;
foreach *section* S_i **do**
    Extract section text;
    Remove redundant and noisy content;
    Identify key sentences using semantic ranking;
    Extract
        Keywords;
        Key concepts;
        Important entities;
        Domain-specific terms;
    Generate section summary using LLM;
    Generate concise abstract of the section;
    Compute summary quality score;
    Associate summary with parent section;
    Update Metadata
        Section ID;
        Summary;
        Keywords;
        Summary length;
    Store summary information in JSON
        Summary text;
        Key concepts;
        Important entities;
        Confidence score;
Validate generated summaries
    Summary completeness;
    Context preservation;
    Factual consistency;
    Length constraints;
if *Validation succeeds* **then**
    Approve section summaries;
else
    Regenerate summary using fallback strategy;
Forward summary repository to Metadata Fusion Agent;
Return section summary repository T;

Algorithm 18: Hallucination Detection Agent

Input: G: Generated content, D: Source document content
Output: H: Hallucination validation report

Receive generated content G and source content D;
Initialize hallucination repository $H \leftarrow \{\}$;
foreach *generated item* g_i **do**
    Retrieve corresponding source content;
    Compute semantic similarity between g_i and source;
    Verify factual consistency
        Entities;
        Numbers;
        Dates;
        Tables;
        Figures;
        Equations;
    Detect unsupported statements;
    Identify omitted critical information;
    Compute hallucination confidence score

$$HScore = 1 - Similarity(g_i, D_i)$$

    if $HScore \leq \tau$ **then**
        Mark content as Verified;
    else
        Flag potential hallucination;
        Request content regeneration;
    Store validation result
        Content ID;
        Validation status;
        Confidence score;
        Detected issues;
    Update Metadata and JSON;
Generate hallucination validation report;
Forward verified content to Confidence Scoring Agent;
Return hallucination validation report H;

Algorithm 19: Confidence Scoring Agent

Input: A: Outputs from all processing agents
Output: C: Confidence score repository

Receive outputs from all agents A;
Initialize confidence repository $C \leftarrow \{\}$;
foreach *document object* o_i **do**
    Collect confidence scores from
        OCR Quality Agent;
        Layout Detection Agent;
        Section Identification Agent;
        Table Analysis Agent;
        Figure Understanding Agent;
        Equation Analysis Agent;
        Entity Extraction Agent;
        Text Summarization Agent;
        Hallucination Detection Agent;
    Compute overall confidence score

$$C_i = \sum_{k=1}^{n} w_k \times c_k$$

    where c_k is the confidence score of agent k and w_k is its corresponding weight.
    Assign confidence level
        High;
        Medium;
        Low;
    Update Metadata
        Object ID;
        Confidence score;
        Confidence level;
    Store confidence information in JSON;
Compute document-level confidence score;
Validate confidence distribution
    Missing scores;
    Invalid values;
    Score consistency;
if *Validation succeeds* **then**
    Approve confidence repository;
else
    Recompute inconsistent scores;
Forward confidence repository to Metadata Fusion Agent;
Return confidence repository C;

Algorithm 20: Metadata Fusion & JSON Appending Agent

Input: A: Validated outputs from all agents
Output: M: Metadata file, J: Structured JSON document

Receive validated outputs from all processing agents;
Initialize Metadata $M \leftarrow \{\}$;
Initialize JSON $J \leftarrow \{\}$;
Create document-level metadata
    Document ID;
    Document Type;
    Document Title;
    Total Pages;
    Processing Timestamp;
foreach *section* S_i **do**
    Append section metadata
        Section ID;
        Section Title;
        Summary;
        Keywords;
        Confidence Score;
    Append extracted objects
        Tables;
        Figures;
        Equations;
        Entities;
        Abbreviations;
        Cross References;
    Append complete section text to JSON;
    Append object descriptions to JSON;
    Append relationships and metadata links;
Append document statistics
    Total Sections;
    Total Tables;
    Total Figures;
    Total Equations;
    Total Entities;
Validate Metadata and JSON schema;
if *Validation succeeds* **then**
    Export Metadata file;
    Export JSON document;
else
    Flag inconsistencies for manual review;
Return Metadata file M and Structured JSON document J;

Algorithm 21: Global Validation Agent

Input: M: Metadata file, J: Structured JSON document
Output: V: Validation report

Receive Metadata M and JSON document J;
Initialize validation report $V \leftarrow \{\}$;
Validate document integrity
    Document information;
    Section hierarchy;
    Page numbering;
    Document statistics;
Validate extracted objects
    Tables;
    Figures;
    Equations;
    Entities;
    Abbreviations;
    Cross references;
Validate Metadata consistency
    Section summaries;
    Keywords;
    Confidence scores;
    Object associations;
Validate JSON schema
    Required fields;
    Object hierarchy;
    Data types;
    Duplicate entries;
Verify cross-document consistency
    Broken references;
    Missing objects;
    Orphan sections;
    Invalid relationships;
Compute document quality score

$$Q = \sum_{i=1}^{n} w_i q_i$$

Generate validation statistics
    Validation accuracy;
    Total errors;
    Warnings;
    Passed checks;
if *Validation succeeds* **then**
    Approve Metadata and JSON;
    Export final document package;
else
    Generate validation report;
    Recommend reprocessing of failed components;
Return validation report V;